

Quantum Spatial Mapping for Quantum Computing

A plain guide to layouts, connectivity, routing, error-correcting grids and
what it means to map a quantum processor

Contents

Preface	1
Chapter 1. Where Is a Qubit?	2
Two meanings of one word	2
Information without a postcode	3
Why location matters at all	3
Chapter 2. The Chip as a Neighbourhood	4
Superconducting circuits: a fixed street plan	4
Trapped ions: a crowded room	5
Neutral atoms: furniture that can be rearranged	5
What all of these have in common	5
Chapter 3. Coupling Maps	7
Reading a coupling map	7
Common shapes	7
Why which qubit sits next to which matters	8
Chapter 4. Moving Information: Swaps and Routing	9
The swap gate	9
Walking a qubit across the chip	9
Routing as a search	10
Initial placement	10
Gates across a gap without moving anything	11
Chapter 5. Swap Networks and Planned Movement	12
The problem of everyone meeting everyone	12
The odd-even pattern	12
Why this is spatial thinking	13
A twist with fermions	13
Chapter 6. Mapping a Device's Noise	14
What goes wrong	14
How the measurements are made	15
The calibration map	15
Using the noise map when mapping a circuit	15

Chapter 7. Compiling a Circuit onto a Real Chip	17
The abstract program	17
Step one: translating into native gates	17
Step two: choosing a layout	18
Step three: routing	18
Step four: optimisation	18
Step five: scheduling	19
Chapter 8. Errors as Defects on a Grid: The Surface Code	20
Why error correction is hard for quantum information	20
The grid	20
Errors as defects	21
Decoding: reading the map	21
When decoding goes wrong	22
The threshold	22
What has been demonstrated	22
Computing with patches	23
Chapter 9. What “Mapping” Can and Cannot Mean	24
Established meanings	24
What mapping cannot mean	25
Speculative territory	26
Chapter 10. The Quantum Spatial Mapping Project, Seen from the Chip	27
What the page says	27
The three expressions	28
Where the project meets a quantum computer	29
Closing	31
Further Reading	32

Preface

This is a book about where things are inside a quantum computer, and about why that question is both more important and less mysterious than it first sounds. A quantum processor is a physical object. Its qubits sit at particular places on a chip, in a vacuum chamber or in a row of laser beams, and only some of them can talk directly to one another. A large share of the practical work of quantum computing consists of coping with that fact: choosing which physical qubit should hold which piece of a calculation, moving information around when two distant qubits need to interact, measuring how well each part of the device behaves, and arranging error correction so that faults show up as visible marks on a grid. All of that is ordinary engineering, and it is a good place for a curious reader to start.

Most of what follows is established. The descriptions of qubit technologies, coupling maps, swap gates, routing, swap networks, noise characterisation, compilation and the surface code reflect mainstream practice as it is taught in textbooks and used on real machines. Where I mention a particular device or experiment, it is one that has been publicly described by the people who built it. I have tried not to quote precise performance figures except where they are widely reported and unlikely to mislead, because these numbers change quickly and differ from machine to machine.

Some of what follows is interpretation, and a small part is speculation. Chapter 9 asks what the word “mapping” can sensibly mean in this setting, and it marks clearly where the discussion leaves engineering and enters the territory of ideas that have not been tested. Chapter 10 discusses the Quantum Spatial Mapping website, which describes itself as “a playful, evolving proposal”. I quote only what the page itself says, I treat it as the open invitation it presents itself as, and I offer some gentle corrections to its mathematics because the page asks for them.

This book follows an earlier one, *Quantum Spatial Mapping and Quantum Teleportation*, which covered qubits, entanglement, the no-cloning theorem, teleportation and the idea of mapping space with quantum information. You do not need to have read it. Where an idea from the earlier book is needed, I explain it again briefly, but I have tried not to retell the same stories. The emphasis here is on the computer itself: the chip, its wiring diagram, its flaws, and the software that fits a calculation onto it. No mathematics beyond simple arithmetic is required, though a few formulas appear in the final chapter for readers who want them.

Chapter 1. Where Is a Qubit?

Ask someone to picture a quantum computer and they will often describe a glittering golden chandelier hanging inside a metal cylinder. That image is real enough. It is the refrigerator that cools one popular kind of processor to a small fraction of a degree above absolute zero, and the chip itself sits at the bottom, about the size of a postage stamp. But the chandelier tells you very little about the calculation. To understand how a quantum computer is organised, it helps to start with a distinction that sounds pedantic and turns out to matter a great deal: the difference between a qubit as a piece of information and a qubit as a physical object.

Two meanings of one word

In the textbooks, a qubit is a unit of quantum information. It is described by two possible basic states, usually written $|0\rangle$ and $|1\rangle$, and by the rule that it can also be in a weighted combination of both, called a superposition. When it is measured, it gives one of the two answers, with probabilities set by those weights. Several qubits together can be entangled, which means that their joint state cannot be described by listing a separate state for each one. None of this description says anything about where the qubit is. A qubit in this sense is like a number in a spreadsheet cell: it has a value, and the rules say how the value can change, but it is not made of anything in particular.

On a real machine, a qubit is also a physical system that carries that information. In a superconducting processor it is a tiny circuit, a loop of metal with a junction in it, whose lowest two energy levels serve as $|0\rangle$ and $|1\rangle$. In a trapped-ion machine it is a single charged atom held in place by electric fields, with two of its internal energy levels playing the same role. In a neutral-atom machine it is an uncharged atom held in the focus of a laser beam. In a photonic machine it may be a single particle of light travelling along a waveguide. Each of these physical qubits does have a location. You could, in principle, point to it.

The two meanings are easy to blur, and the blurring causes confusion. It is tempting to think of a qubit as a little place, a box in which a quantum state lives, and to think of a quantum computer as a set of such boxes arranged in space. That picture is half right. The boxes exist, and their arrangement matters, as most of this book will show. But the information is not tied to any particular box. A quantum state can be moved from one physical qubit to another, by operations that will be described in Chapter 4, and the calculation does not care which box it was in. What the calculation cares

about is which pieces of information can be brought together, and how easily.

Information without a postcode

If you ask where the information of “logical qubit number 3” is during a calculation, the honest answer is that it depends on the moment. At the start it may be in the physical qubit at the top-left corner of the chip. After a few swaps it may be in a qubit three positions away. In a machine where atoms are physically shuffled around, the carrier itself may have moved. And once qubits are entangled, the information is not located in any single carrier at all: it is shared between them, held in the pattern of correlations rather than in one place.

So a quantum program does not describe a set of places. It describes a set of logical qubits, numbered or named, and a sequence of operations on them. The programmer writes “apply a controlled-NOT gate between qubit 0 and qubit 4” without needing to know that, on the actual chip, those two qubits might be far apart. Someone or something has to decide where each logical qubit will physically live, and what to do when the program asks two distant ones to interact. That decision is called mapping, and it is the first of several meanings of that word in this book.

Why location matters at all

If information can be moved, why does the physical arrangement matter? Because moving it costs something. Every operation on a real quantum computer has a small chance of going wrong, and every moment a qubit spends waiting, it slowly loses its quantum character through contact with its surroundings. A calculation that requires many moves takes longer and suffers more errors than one that requires few. On today’s machines the difference can decide whether a calculation produces a useful answer or noise.

There is also a deeper reason. The physical interactions that make two-qubit gates possible are local. Two superconducting circuits interact because they are wired together through a small capacitor or a tunable coupler. Two ions interact through their shared motion in a trap. Two neutral atoms interact when they are close enough for one to feel the other’s electric field when both are excited. Nature does not provide a way to make two arbitrary, widely separated objects interact strongly on demand. So every quantum processor has a pattern of who can talk to whom, fixed by its physics and its engineering, and every calculation has to be fitted to that pattern.

This is where spatial ideas meet quantum computing, in a very down-to-earth way. A quantum processor is a neighbourhood. Some qubits are next-door neighbours and can work together directly. Others live across town and can only cooperate through intermediaries. Much of the craft of using these machines lies in planning the neighbourhood well and in routing messages through it efficiently.

Chapter 2. The Chip as a Neighbourhood

Different kinds of quantum hardware produce very different neighbourhoods. This chapter visits three of the main families of qubit technology and describes, in each case, how qubits are placed and how they come to interact. The details of the physics are left out. The aim is to see what the arrangement looks like and why it looks that way.

Superconducting circuits: a fixed street plan

Superconducting qubits are built using the same broad techniques as ordinary computer chips: thin films of metal are patterned on a substrate, usually silicon or sapphire, to make tiny circuits. Once the chip is made, its layout is fixed. Each qubit occupies a particular spot, and each connection between two qubits is a physical structure, often a small capacitor or a separate tunable element called a coupler, placed between them. If two qubits were not joined when the chip was designed, they cannot be joined later.

This produces a street plan that is literally etched into the device. Designers face a set of competing pressures. More connections per qubit make calculations easier to fit, because more pairs can interact directly. But every connection adds wiring, takes up space, and creates opportunities for unwanted interaction, known as crosstalk, in which an operation on one qubit disturbs its neighbours. Each qubit also needs control lines and readout lines running to it from the outside world, and those lines have to be squeezed in among everything else. In practice, superconducting processors give each qubit somewhere between two and four direct neighbours.

Two well-known layouts illustrate the range. Google's Sycamore processor, described in 2019, arranged its qubits in a square grid tilted on its side, with each qubit joined to up to four neighbours through tunable couplers. IBM's processors have for several years used a pattern the company calls heavy-hex: a lattice of hexagons with extra qubits placed along the edges, so that every qubit has either two or three neighbours. The heavy-hex pattern gives up some connectivity in exchange for less crosstalk and simpler frequency planning, a trade-off its designers have explained publicly. Neither choice is simply better. Each suits certain uses, and each shapes what calculations run well.

Trapped ions: a crowded room

In a trapped-ion machine, the qubits are individual atoms with one electron removed, held in a line by electric fields in a vacuum. Because the ions are all charged, they push on one another, and when one is nudged with a laser the whole chain can vibrate together. That shared vibration acts as a bus: by carefully timed laser pulses, any two ions in the chain can be made to interact, regardless of their position in the line.

So within a single chain, the neighbourhood is a crowded room in which everyone can talk to everyone. This is often called all-to-all connectivity, and it removes most of the routing problems discussed later in this book. It comes with its own costs. Gates on trapped ions are typically slower than on superconducting circuits. And a single chain cannot grow indefinitely: as more ions are added, the chain's vibrations become harder to control precisely, and gates become slower or less accurate.

The usual answer is to build several smaller zones on one trap and move ions between them. In designs often described as a quantum charge-coupled device, electrodes shuttle ions from a storage area to an interaction area, bring two ions together for a gate, and carry them away again. Here the neighbourhood is not fixed at all: the qubits themselves travel. Routing becomes a matter of transport timing rather than of choosing a path through a fixed graph.

Neutral atoms: furniture that can be rearranged

Neutral-atom machines hold individual atoms in tightly focused laser beams called optical tweezers. Hundreds of such tweezers can be arranged in a two-dimensional pattern, and the atoms in them can be moved by steering the beams. To make two atoms interact, they are brought close together and excited to a very high energy level, known as a Rydberg state, in which the atom becomes enormously large and strongly affects any other excited atom nearby.

This gives a neighbourhood that can be rearranged during a calculation. In demonstrations published since about 2022, groups have moved whole rows of atoms in parallel, bringing selected pairs into contact, applying gates and then moving them on. The interaction is still local: atoms must be near each other to interact. But the map of who is near whom is no longer fixed by manufacture. It is chosen, and can be changed, while the program runs.

What all of these have in common

The same is true of photonic machines, which guide particles of light through networks of waveguides. Despite their differences, every one of these technologies has a pattern of allowed interactions, and that pattern is constrained by physical locality. In some machines the pattern is fixed by manufacture. In others it can be reconfigured, at a cost in time. In still others it is nearly complete within a module but sparse between modules. In each case there is a set of pairs of qubits that can interact cheaply now, and a set that can interact only after something has been moved.

That set of cheap pairs can be drawn as a diagram: dots for qubits, lines for direct interactions. Engineers call this diagram a coupling map, or sometimes a connectivity graph. It is the single most useful picture for understanding how a calculation will fit onto a particular machine, and it is the subject of the next chapter.

Chapter 3. Coupling Maps

A coupling map is a simple thing to draw and a surprisingly rich thing to think about. It is a graph, in the mathematician's sense: a collection of points, called vertices or nodes, joined by lines, called edges. Each vertex is a physical qubit. Each edge says that a two-qubit gate can be applied directly between the two qubits it joins. That is all. The map does not say how far apart the qubits are on the chip in millimetres, nor what colour the wires are, nor how good each connection is. It records only who can interact with whom.

Reading a coupling map

Suppose you are handed a coupling map of a small device with seven qubits, drawn as follows. Qubits 0, 1, 2, 3 and 4 form a line, each joined to the next. Qubit 5 is attached to qubit 1, and qubit 6 is attached to qubit 3, like two short branches hanging off the line. What can you learn just by looking?

First, the degree of each qubit, which is simply how many edges touch it. Qubits 0, 4, 5 and 6 have degree one: each has a single neighbour. Qubits 1 and 3 have degree three. Qubit 2 has degree two. A qubit with high degree is a well-connected junction; a qubit with degree one is a cul-de-sac.

Second, the distance between any two qubits, measured as the smallest number of edges you must cross to get from one to the other. Qubit 0 and qubit 1 are at distance one. Qubit 5 and qubit 6 are at distance four: from 5 to 1, 1 to 2, 2 to 3, 3 to 6. The largest such distance on the whole map is called its diameter. For this small device the diameter is four, achieved for example between qubit 0 and qubit 6, or between qubit 5 and qubit 6. (From 0 to 4 along the line is also four.)

Third, bottlenecks. Every path from the left half of this device to the right half passes through qubit 2. If qubit 2 performs badly, or if many interactions need to cross from one side to the other at the same time, qubit 2 will become a point of congestion. Graph theorists call such a vertex a cut vertex, and they have a whole vocabulary for measuring how well a graph holds together. For our purposes the plain idea is enough: some qubits sit on many shortest paths and so matter more than others.

Common shapes

A few families of graph keep turning up.

The line, or linear chain, is the simplest: each qubit has two neighbours except the two at the ends. It is easy to build and easy to analyse, and it is the worst case for routing, because two qubits at opposite ends are as far apart as possible.

The square grid gives each interior qubit four neighbours. The worst distance on a square grid of n qubits grows roughly in proportion to the square root of n , which is far better than the line, where it grows in proportion to n itself. A grid of 100 qubits laid out in ten rows of ten has a diameter of 18; a line of 100 qubits has a diameter of 99. The grid is also the natural home of the surface code, which is one strong reason designers favour it.

The heavy-hex lattice, mentioned in the previous chapter, has degree two or three everywhere. It sits between the line and the grid in connectivity, and was chosen to reduce errors that arise when neighbouring qubits interfere with each other.

The complete graph, in which every qubit is joined to every other, is what a single chain of trapped ions approximates. Its diameter is one. There is nothing to route.

Modular designs combine these: dense connectivity within small groups, joined by a few longer-range links between groups. Those links might be special couplers on a chip, a microwave cable between chips in the same refrigerator, or an optical fibre between separate machines. Long links are generally slower and noisier than local ones, so a modular coupling map is best thought of as having edges of different weights.

Why which qubit sits next to which matters

Imagine a calculation in which qubit A must interact with qubits B, C, D and E in turn. On a grid you might place A in the middle with B, C, D and E as its four neighbours, and everything happens directly. On heavy-hex, no qubit has four neighbours, so at least one of those interactions will need a detour. On a line, A can have only two neighbours, so two of the four interactions need detours. On a trapped-ion chain, all four can happen directly.

This simple example captures the main idea. Every calculation has its own interaction pattern, a graph of which logical qubits need to interact and how often. Every device has its coupling map. The work of mapping is the work of laying the first graph onto the second as well as possible. When the calculation's graph fits inside the device's graph, as a smaller shape fits inside a larger one, every interaction can be done directly. When it does not fit, which is most of the time, extra work is needed. Finding the best possible fit, including the extra moves, is known to be a computationally hard problem in general, so practical tools rely on clever approximations and search methods that usually find a good answer rather than a guaranteed best one.

Chapter 4. Moving Information: Swaps and Routing

When a calculation asks two qubits to interact and the coupling map has no edge between them, something has to give. Either the information in one of them is moved until the two are neighbours, or the interaction is carried out indirectly through the qubits in between. This chapter describes the main ways of doing that, beginning with the most common, the swap.

The swap gate

A swap gate takes two neighbouring qubits and exchanges their states. If the first held state ψ and the second held state ϕ , then afterwards the first holds ϕ and the second holds ψ . It sounds trivial, and in ordinary computing it would be: you would just copy each value into a temporary register. In quantum computing it cannot be done by copying, because unknown quantum states cannot be copied. Instead, the swap is built from genuine quantum operations that exchange the states directly. A standard construction uses three controlled-NOT gates in a row, the middle one pointing the opposite way from the outer two.

The important point is that a swap is not free. On a device whose native two-qubit gate is a controlled-NOT, or something equivalent, one swap costs roughly three two-qubit gates. Two-qubit gates are usually the slowest and least reliable operations on a device. So every swap adds time and adds roughly three opportunities for error. On some hardware a swap can be implemented a little more cheaply, or merged with a neighbouring gate in the program, but the general lesson holds: moving information costs about as much as several pieces of real computation.

Walking a qubit across the chip

Return to the simplest coupling map, a line of five qubits labelled 0 to 4. Suppose the program needs a controlled-NOT between the information held in qubit 0 and the information held in qubit 4. These are at distance four. One plan is to walk the first state along the line: swap 0 with 1, then 1 with 2, then 2 with 3. After three swaps the state that started in qubit 0 is in qubit 3, right next to qubit 4, and the controlled-NOT can be applied directly. The other qubits' states have each moved one place to the left in the process, which matters if they are part of the calculation too.

Another plan is to move both ends towards the middle: swap 0 with 1, and at the same time swap 4 with 3. Now the two states are in qubits 1 and 3, still at distance two, so one more swap is needed, giving three swaps again. The total number of swaps is the same, but two of them happened at the same time, so the calculation takes less wall-clock time. In general, bringing together two qubits at distance d along a path needs $d - 1$ swaps, and the art lies in choosing which swaps to do in parallel and which to share between several upcoming gates.

That last point is where the real gains are. A program rarely contains just one long-range gate. If the next gate after the one above also needs qubit 3's new resident, the swaps already done may have helped it. If the next gate needs the qubit that was just pushed out of the way, they may have hurt. A good router looks ahead.

Routing as a search

The task of inserting swaps so that every two-qubit gate in a program acts on neighbours is called routing. It has a clear structure. The program is a list of gates in an order, with some freedom: gates on different qubits can often be reordered or done at the same time. At each moment, the router knows where every logical qubit currently lives on the chip. It looks at the gates that are ready to run. Those whose qubits are already neighbours can be done immediately. For the rest, it chooses some swaps that bring them closer, updates its record of who lives where, and repeats.

Choosing the swaps is the hard part. A greedy rule, such as always picking the swap that most reduces the total distance for the gates waiting right now, is quick but short-sighted. A more careful rule also weighs the gates coming soon after, perhaps with a discount for gates further in the future. A still more careful approach explores several possible sequences and picks the best, at the cost of more computing time. Published methods span this whole range. One widely used heuristic, known as SABRE and described in 2019, combines a look-ahead cost of this kind with a trick of running the routing forwards and backwards through the program to choose a good starting placement. Variants of it are used in widely available open-source compilers.

Initial placement

Routing is only half of the job. Before the first gate runs, each logical qubit must be assigned to a starting physical qubit. This is called the initial layout or initial placement, and it can make a large difference. A good placement puts logical qubits that interact frequently onto neighbouring physical qubits, so that many gates need no swaps at all. A poor placement can double or triple the number of swaps needed later.

Finding a good placement is itself a version of the graph-fitting problem from the previous chapter. One approach is to look for an exact fit, an embedding of the program's interaction graph into the coupling map, and use it if one exists. Another is to try many random placements, route each, and keep the best. Another is to place

the most connected logical qubits first, on the best-connected physical qubits, and fill in around them. All of these are used. None is best in every case.

Gates across a gap without moving anything

Swapping is not the only way to make distant qubits interact. One alternative, sometimes called a bridge, performs a controlled-NOT between two qubits that share a common neighbour, using four controlled-NOT gates through the middle qubit, and leaves all three qubits where they were. It costs more gates than a single swap followed by a direct gate, but it does not disturb the layout, which can be worth it when the middle qubit's resident is needed where it is. A more striking alternative uses measurement: spare qubits along a path are entangled, measured, and the results sent through ordinary classical wiring to correct the far end, a close relative of teleportation. As with teleportation, the operation is not complete until those classical signals arrive, so nothing outruns light.

Chapter 5. Swap Networks and Planned Movement

Routing, as described in the last chapter, is often reactive: the router looks at what the program needs next and inserts swaps to make it possible. For some calculations there is a better way. If the pattern of interactions is known in advance and has a regular structure, one can design a fixed schedule of swaps that brings every required pair together exactly when needed. Such a schedule is called a swap network. It is one of the clearest examples of spatial reasoning in quantum computing, and it rewards a closer look.

The problem of everyone meeting everyone

Consider a calculation in which every logical qubit must interact once with every other. This happens, for example, in some methods for simulating molecules, where the electrons in each orbital interact with those in every other orbital, and in some optimisation algorithms applied to problems where every variable is linked to every other. With n qubits there are $n(n - 1)/2$ pairs. On a fully connected machine they could all be done directly. On a line, most pairs are not neighbours.

A reactive router would handle the pairs one at a time, moving qubits back and forth, and could easily end up wasting many swaps undoing earlier moves. There is a much tidier solution, and it comes from an old idea in sorting.

The odd-even pattern

Lay n qubits in a line. In the first step, pair up positions 1 and 2, 3 and 4, 5 and 6, and so on. For each pair, apply the interaction the calculation needs, and then swap the two qubits. In the second step, pair up positions 2 and 3, 4 and 5, and so on, and do the same. Alternate these two kinds of step. After n steps, the order of the qubits along the line has been exactly reversed, and, remarkably, every pair of qubits has been neighbours exactly once along the way.

You can check this with four qubits labelled A, B, C, D. Start with the order A B C D. Step one pairs A with B and C with D; after swapping, the order is B A D C. Step two pairs the middle two, A and D; after swapping, B D A C. Step three pairs B with D and A with C; after swapping, D B C A. Step four pairs B with C; after swapping, D C B

A. The pairs that met were AB, CD, AD, BD, AC and BC, which is all six pairs of four qubits, each exactly once. The order has been reversed.

This works for any number of qubits, and it takes n layers of work on a line with no wasted moves. Because the interaction and the swap act on the same pair, they can often be merged into a single combined operation, so the swaps cost much less than they would if added separately. The pattern is borrowed from a sorting method known to computer scientists as odd-even transposition sort, in which a list is sorted by repeatedly comparing and exchanging neighbours in this alternating way.

Why this is spatial thinking

The swap network turns a problem about who must meet whom into a problem of planned motion. The qubits are like dancers in a line who pass along it, each greeting every other dancer exactly once as they go by. The schedule does not depend on any search; it is fixed by the geometry of the line. And it is optimal in a meaningful sense for a line: since each layer can bring together at most about $n/2$ new pairs, and there are about $n^2/2$ pairs in total, around n layers is the least one could hope for.

A twist with fermions

There is a subtlety worth mentioning, because it shows how spatial ordering can carry physical meaning in a simulation. When a quantum computer is used to simulate electrons, it has to respect a rule that electrons obey: exchanging two of them changes the sign of the overall state. A common way of encoding electrons into qubits, the Jordan-Wigner encoding, handles this rule by putting the electrons' orbitals in a line and keeping track of the ordering. Interactions between orbitals far apart in this line become long, expensive chains of operations.

A neat solution, published in 2018, is the fermionic swap network. It replaces the ordinary swap with a version that also applies the required sign change, so that swapping two qubits genuinely swaps the order of the two orbitals in the encoding. After that, the odd-even pattern brings every pair of orbitals into adjacency, and every interaction becomes short and cheap. The line of qubits is still a line, but now the ordering along it is part of the physics being simulated, and moving qubits around is equivalent to relabelling which orbital is where. This is a clean example of a layout that is not just a hardware inconvenience but a carefully designed part of the algorithm.

Chapter 6. Mapping a Device's Noise

A coupling map tells you which qubits can interact. It does not tell you how well. On every real device, some qubits are better than others, some connections are more reliable than others, and the whole picture shifts from day to day. Characterising those differences is a large and careful part of running a quantum computer. The result, a kind of map of the device's imperfections, is one of the most practically useful senses in which a quantum processor gets mapped.

What goes wrong

Errors on a quantum computer come from several sources, and it helps to name the main ones.

A qubit that has been put into its higher-energy state tends, over time, to fall back to its lower-energy state, giving up its energy to its surroundings. The typical time this takes is called the relaxation time, conventionally written T_1 . A qubit in a superposition also tends to lose the delicate phase relationship between its two parts, even without losing energy. The typical time for that is called the dephasing or coherence time, written T_2 . Both times vary from qubit to qubit, and both set a limit on how long a calculation can run before the information degrades.

Gates are imperfect. A single-qubit gate might rotate the state by slightly too much or too little. A two-qubit gate might leave a small unwanted interaction behind. These faults are usually summarised as an error rate for each gate on each qubit or pair of qubits.

Measurement is imperfect too. When a qubit is read out, there is some chance that a $|0\rangle$ is reported as 1 or a $|1\rangle$ as 0. This readout error is often larger than the error of a single-qubit gate.

Then there is crosstalk: an operation on one qubit disturbing others nearby, or two gates done at the same time interfering with each other. Crosstalk is a genuinely spatial kind of error, because it depends on which qubits are close to which, physically or in frequency.

Finally there is leakage, in which a qubit leaves its two computing levels for a higher one, and there are rarer bursts of errors across many qubits at once, which some

groups have traced to high-energy radiation striking the chip.

How the measurements are made

Each of these quantities has standard measurement methods. Relaxation is measured by putting a qubit in its higher-energy state, waiting a chosen time, and measuring how often it is still there. Repeating this for many waiting times gives a decay curve whose shape reveals T_1 . Coherence is measured with sequences in which a superposition is created, left alone or gently manipulated for a chosen time, and then converted back and measured; the fading of the resulting interference pattern reveals T_2 .

Gate errors are commonly measured by a family of methods called randomised benchmarking. The idea is pleasingly indirect. Instead of trying to inspect a single gate in detail, you apply long random sequences of gates chosen so that, if everything were perfect, the sequence as a whole would do nothing. You then measure how often the qubit fails to return to its starting state, and how that failure rate grows with the length of the sequence. The rate of growth gives an average error per gate that is largely unaffected by the errors in preparing and measuring the qubit, which are confined to the beginning and end. Variants of the method isolate particular gates, or measure errors on pairs of qubits.

The calibration map

Operators of cloud-accessible quantum computers usually publish a summary of these numbers for each device, refreshed regularly. It is typically shown as the coupling map with colours: each qubit coloured by its readout error or coherence time, each edge coloured by its two-qubit gate error. A glance shows where the good regions are and where the trouble spots lie. Often one or two qubits are markedly worse than the rest, and occasionally a qubit or connection is marked as unusable.

This is a map in a straightforward sense: a spatial picture of a property that varies from place to place. It is closer to a weather map than to a street map, because it changes. Superconducting qubits in particular can drift as tiny defects in their materials shift about, so the best operating frequency for a qubit today may not be the best tomorrow. Devices are recalibrated regularly, sometimes several times a day, and the calibration map is only a snapshot.

Using the noise map when mapping a circuit

Once the device's noise is mapped, the compiler can use it. Instead of treating every edge of the coupling map as equal, it can give each edge a cost reflecting how error-prone it is, and look for placements and routes that minimise total expected error rather than the raw number of swaps. A program might be placed on a slightly less convenient region of the chip because that region's qubits are much better. A route might take a longer path around a bad connection rather than straight through it. Readout could be assigned to the qubits with the most reliable measurement.

These noise-aware strategies are well established and widely used, though they come with a caveat. The calibration data are averages, measured under conditions that may not match the actual program. A qubit whose gates are fine in isolation may suffer badly from crosstalk when its neighbours are busy. So the most careful approaches measure errors under conditions that resemble the program, or run small test circuits first.

Chapter 7. Compiling a Circuit onto a Real Chip

The preceding chapters introduced the pieces separately: the coupling map, routing, swap networks and noise maps. This chapter puts them together by following a single small program from its abstract form to something a real device could run. The tools that do this are usually called compilers or, in some popular software, transpilers. The steps described here are common to most of them, though different tools organise and name them differently.

The abstract program

A quantum program is usually written as a circuit: a set of horizontal lines, one per logical qubit, with gates drawn along them from left to right in the order they are applied. Single-qubit gates sit on one line. Two-qubit gates join two lines with a vertical stroke. At the end, measurements turn each qubit's state into a classical bit.

Take a small example with four logical qubits, called a, b, c and d. The program prepares a superposition on a, then applies controlled-NOT gates from a to b, from a to c and from a to d, and finally measures all four. If it ran perfectly, this program would produce an entangled state in which all four qubits agree: measuring them would give either 0000 or 1111, each about half the time. It is a standard demonstration, and its interaction pattern is a star: a interacts with each of the other three, and they do not interact with each other.

Step one: translating into native gates

Real hardware does not offer every gate a textbook describes. Each device has a small set of native operations, typically a few single-qubit rotations and one kind of two-qubit gate. The first step of compilation is to rewrite every gate in the program in terms of these. The superposition-making gate might become a pair of rotations. The controlled-NOT might become the device's native two-qubit gate surrounded by single-qubit rotations. Sometimes this step is done after routing rather than before, but it has to be done somewhere.

Step two: choosing a layout

Next, the compiler must decide which physical qubit will hold each logical qubit. Suppose the device is the seven-qubit one from Chapter 3: a line 0, 1, 2, 3, 4, with qubit 5 hanging off qubit 1 and qubit 6 hanging off qubit 3.

The program's interaction graph is a star with *a* at the centre and three spokes. The device has two qubits of degree three, qubits 1 and 3. Placing *a* on qubit 1, with *b*, *c* and *d* on qubits 0, 2 and 5, fits the star exactly. Every controlled-NOT in the program now acts on neighbours, and no routing is needed at all.

Now suppose the noise map says that qubit 5 has poor readout, or that the connection between 1 and 5 is unreliable. The compiler might instead place *a* on qubit 3, with *b*, *c* and *d* on 2, 4 and 6, if that region is better. Or, if both regions have a problem, it might choose a layout on the line that is not a perfect fit and accept one swap in exchange for using better qubits. This is the kind of trade-off a noise-aware compiler makes routinely.

Step three: routing

Suppose for illustration that the device were simply a line of four qubits, with no branches. Then no qubit has three neighbours, and the star cannot fit. Place *a* on qubit 1, with *b* on 0, *c* on 2 and *d* on 3. The gates from *a* to *b* and from *a* to *c* are direct. The gate from *a* to *d* is not, because qubits 1 and 3 are at distance two. The router has choices: swap qubits 2 and 3, so that *d* moves next to *a*, at the cost of moving *c* away, which no longer matters because *c*'s gate has been done; or swap 1 and 2, moving *a* next to *d*. Either costs one swap, roughly three two-qubit gates. A cleverer compiler might notice something about this particular program: after the gate from *a* to *c*, the two qubits agree in every branch of the superposition (both 0 or both 1, though this is entanglement, not a copy of an unknown state), so a controlled-NOT from *c* to *d* has the same effect as one from *a* to *d*. That replaces the long-range gate with a direct one at no cost. Recognising such equivalences is part of optimisation, the next step.

Step four: optimisation

After translation and routing, the circuit usually contains redundancies. Two rotations in a row on the same qubit can be merged into one. A gate followed by its own inverse can be removed. A swap followed by a two-qubit gate on the same pair can sometimes be combined into a single, cheaper operation. More advanced passes look for larger patterns that can be resynthesised more efficiently, or exploit facts about the state at a given point in the program, as in the example above. Optimisation passes are often run repeatedly until nothing more improves.

Step five: scheduling

The compiler now has a list of native gates on physical qubits. It must decide exactly when each one happens. Gates on separate qubits can often run at the same time, which shortens the program and so reduces the time qubits spend decaying. But running some gates at the same time causes crosstalk, so the scheduler may deliberately separate them. Qubits that are idle while others work may be given short sequences of pulses, a technique called dynamical decoupling, which help protect them from slow drifts in their environment. Measurements also take time and can disturb neighbouring qubits, so they too need scheduling.

Chapter 8. Errors as Defects on a Grid: The Surface Code

Everything so far has been about fitting calculations onto imperfect hardware. Error correction is about making the hardware behave, from the program's point of view, as though it were nearly perfect. The leading approach for many kinds of hardware is a scheme called the surface code, and it is profoundly spatial. It treats errors as marks that appear on a grid, and it treats correcting them as a kind of map-reading. This chapter explains the idea at a high level. It is not a guide to building one; the details are formidable, and the further reading points to proper accounts.

Why error correction is hard for quantum information

Ordinary error correction often relies on copying. Store three copies of a bit, and if one flips, the majority vote recovers the right value. Quantum information cannot be copied, and measuring a qubit to check on it disturbs it. For a while in the early 1990s, these facts led some to suspect that quantum error correction might be impossible.

The way around both problems, discovered in the mid-1990s, is to spread a single qubit of information across several physical qubits in an entangled pattern, and then to measure not the qubits themselves but certain collective properties of groups of them. These collective properties are chosen so that they reveal whether an error has occurred, and roughly what kind and where, without revealing anything about the encoded information. The measured values are called the syndrome, a term borrowed from medicine: they are symptoms, not the patient.

The grid

In the surface code, physical qubits are arranged on a two-dimensional square grid. Roughly half of them, called data qubits, hold the encoded information. The other half, interleaved among them, are measurement qubits, sometimes called ancillas. Each measurement qubit in the interior of the grid sits among four data qubits, its nearest neighbours, and its job is to check one collective property of those four, over and over again, in rounds.

There are two kinds of measurement qubit, arranged like the two colours of a chessboard. One kind checks for one type of error, bit flips, which turn $|0\rangle$ into $|1\rangle$

and vice versa. The other kind checks for the other basic type, phase flips, which change the sign relating the two parts of a superposition. Any small error on a qubit can be described as a combination of these two, so between them the two kinds of check cover everything.

The crucial feature is that every check involves only nearest neighbours on a grid. That is exactly the kind of interaction a superconducting chip with a square coupling map can provide directly. No long-range gates are needed, and no swaps. This is one of the main reasons the surface code is so popular: it fits naturally onto realistic hardware.

Errors as defects

Now imagine that, in the middle of a long, quiet stretch of rounds, one data qubit suffers a bit flip. The measurement qubits next to it that check for bit flips will notice: their results change from what they were in the previous round. On a grid, a single data qubit in the interior is watched by two checks of each type, and both of the relevant ones flip. If you draw the grid and mark the checks whose results changed, you see two marks, one on each side of the faulty qubit.

These marks are often called defects, and that is the spatial heart of the idea. The surface code turns errors into visible spots on a map. The spots appear at the ends of chains of errors. If two neighbouring data qubits both flip, the check between them sees two changes, which cancel out, and only the checks at the far ends of the pair light up. A longer chain of errors, snaking across the grid, shows up only as two defects at its ends. The middle of the chain is invisible.

Errors in the measurements themselves also happen. A measurement qubit might simply report the wrong value once. Because the checks are repeated round after round, such a fault shows up as a defect that appears and then vanishes in time, rather than in space. So the full picture is a three-dimensional map: two dimensions of the grid, and one of time, with defects scattered through it.

Decoding: reading the map

The task of working out which errors most likely occurred, given the pattern of defects, is called decoding. It is a puzzle about a map. Given a scattering of marks, find the most plausible set of short chains that would have produced them, pairing the marks up. Since errors are rare, short chains are more likely than long ones, so a good guess pairs each defect with a nearby one.

A standard method does exactly this using a classic algorithm from graph theory, minimum-weight perfect matching, which finds the way to pair up all the points that minimises total length. Other decoders use different strategies, some faster, some more accurate, some designed to run on special-purpose hardware quickly enough to keep pace with the device. Speed matters, because the defects keep arriving every round, often every microsecond or so on superconducting hardware, and the decoder must not fall behind.

When decoding goes wrong

Decoding can fail. If errors are frequent, defects become numerous and close together, and the decoder may pair them up wrongly. A wrong pairing, combined with the actual errors, can form a chain that stretches all the way across the grid from one edge to the opposite edge. Such a chain is not just an uncorrected error; it changes the encoded information itself. It is a logical error.

This gives the surface code its central property. A logical error needs a chain of physical errors long enough to cross the grid. The length of the shortest such chain is called the code distance, usually written d , and it is roughly the width of the patch of grid used for one logical qubit. If physical errors are rare enough, the chance of enough of them lining up to cross a distance- d patch falls rapidly as d grows. Make the patch bigger, and the logical qubit gets better. The encoded information lives in a global property of the whole patch, something like whether a chain crosses from one side to the other, which no small, local disturbance can change. This is why the surface code belongs to a family called topological codes, and it is one place where information really is tied to an arrangement in space, though spread across the whole patch rather than located at any point within it.

The threshold

“Rare enough” has a precise meaning. Theory and simulation show that there is a threshold error rate for the physical operations. Below it, enlarging the code suppresses logical errors more and more. Above it, enlarging the code makes things worse, because the extra qubits bring in errors faster than the extra distance can catch them. For the surface code under realistic models of noise, the threshold is around one per cent per operation, a figure that is unusually forgiving among error-correcting codes and is another reason for the code’s popularity.

Being just below threshold is not enough for practical use. To make logical errors truly rare, physical errors need to be well below threshold, and the patch of grid for each logical qubit needs to be large. Estimates for useful, large-scale calculations commonly suggest hundreds or thousands of physical qubits per logical qubit, depending on the hardware and the target. That overhead is one of the main reasons quantum computers capable of the most famous algorithms do not yet exist.

What has been demonstrated

Experiments in recent years have shown the key behaviour. In a result published in 2023, Google’s team reported that a distance-5 surface code on their superconducting processor performed slightly better than a distance-3 code, the first time on their hardware that making the code larger made the logical qubit better. In work described in 2024, using a newer processor, the same group reported a distance-7 code, with logical error rates falling by roughly a factor of two each time the distance increased by two, a clear sign of operating below threshold. Other groups, using trapped ions and neutral atoms, have demonstrated small logical qubits and

operations between them using a variety of codes. These are genuine milestones. They are also early: the logical qubits are few, and their error rates are not yet low enough for long calculations.

Computing with patches

A grid that merely stores information is called a memory. To compute, logical qubits must interact. In the surface code, one widely studied method, called lattice surgery, does this by temporarily merging two neighbouring patches along an edge, measuring joint properties across the seam, and then splitting them again. All of these are spatial operations on a grid, and they bring back the themes of earlier chapters on a larger scale. Logical qubits are laid out on a chip in patches; some patches are neighbours and some are not; interactions between distant patches need corridors of spare grid to pass through; and planning that layout well is again a routing problem.

So the problem of mapping does not disappear with error correction. It moves up a level. Instead of placing single qubits on a coupling map, an error-corrected computer places patches on a floor plan, with lanes between them, and schedules their merging and splitting. Designers of such layouts speak, quite naturally, of tiles, lanes and factories.

Chapter 9. What “Mapping” Can and Cannot Mean

The word “mapping” has appeared in this book in several senses. That is not carelessness; the word genuinely has several established uses in quantum computing, and a few more speculative ones in the wider conversation about quantum physics and space. This chapter separates them. The first part lists the meanings that are standard engineering. The second lists some things that mapping, in this setting, cannot mean. The third, clearly marked, looks at speculative ideas that people sometimes connect with the phrase “quantum spatial mapping”, and says what would be needed to take them further.

Established meanings

Logical-to-physical mapping. The assignment of a program’s logical qubits to a device’s physical qubits, together with the changes to that assignment made by routing as the program runs. This is the most common technical meaning of the word in quantum compilation, and it is a well-defined, well-studied problem. It was the subject of Chapters 3, 4 and 7.

Mapping a problem onto a device’s graph. A broader version of the same idea, used when a problem has its own natural graph of interactions, for example an optimisation problem whose variables are linked in a network. Fitting that graph onto the hardware is called embedding. On quantum annealers, a different style of quantum machine designed for optimisation, the problem graph often does not fit the hardware graph directly, and a common technique represents a single problem variable by a chain of several physical qubits held in agreement, so that the chain can reach all the neighbours the variable needs. This is called minor embedding, after the graph-theory notion of a minor. It is a clear case where one logical object is deliberately spread across a region of the chip.

Mapping a physical problem into qubits. When a quantum computer simulates something else, a molecule, a material, a lattice of interacting particles, the thing being simulated has to be translated into qubits and gates. This translation is also called a mapping or an encoding. The Jordan-Wigner encoding of electrons into qubits, mentioned in Chapter 5, is one example; there are several others, each with its own trade-offs between how many qubits are used and how local the resulting interactions are. Often the simulated system has its own geometry, such as a chain

of atoms or a two-dimensional crystal, and a good mapping tries to keep things that are close in the simulated system close on the chip, so that the simulation needs few long-range operations. This is a genuinely spatial design principle, and an important one.

The surface-code layout. The arrangement of logical patches, lanes and resource factories on an error-corrected machine, discussed at the end of Chapter 8. It is a floor plan in a fairly literal sense.

All of these are real, practical and actively developed. They share a theme: mapping, in quantum computing, is about relations of locality, about who is next to whom, and about how to make the relations a calculation needs fit the relations a device provides.

What mapping cannot mean

Some things are worth ruling out plainly, because the vocabulary invites them.

A qubit is not a point in space. The state of a logical qubit is not stored at a location the way water is held in a cup. On a device it is carried by a physical system that has a location, but the state can be moved, and once qubits are entangled the information is held jointly rather than at any single site. A map of where the physical qubits are is a map of the carriers, not of the information.

A coupling map is not a map of physical space. It records which interactions are possible. It often resembles the chip’s floor plan because interactions are local, but distance on a coupling map counts steps, not millimetres, and nothing in it describes the geometry of the space the chip sits in.

Moving a state is not travelling without crossing the distance. Swaps move information by a chain of local interactions, each taking time. Measurement-based tricks for long-range gates, like teleportation, require classical signals to reach their destination before they are complete. No method of moving quantum information within a computer, or between computers, sends a usable signal faster than light.

A quantum computer does not map space by being quantum. A quantum processor can be used to simulate a physical system that has a geometry, and it can be used, with suitable hardware, as part of a sensor that measures position, rotation or fields very precisely. Those are real applications. But running a calculation on a chip does not, by itself, reveal anything about the geometry of the room it sits in, or of space in general.

Entanglement is not a hidden wire. It is natural to imagine that two entangled qubits are connected by an invisible link that could be used to send messages. They are not. Entanglement gives correlations that show up only when the results of measurements on both sides are compared, and that comparison needs an ordinary channel. This is why entanglement is a resource for computing and communication, consumed by use, rather than a shortcut through space.

Speculative territory

Everything in this section goes beyond established engineering. Some of it touches real research questions; none of it is a demonstrated result.

Space from entanglement. A line of theoretical research, mainly in the study of quantum gravity, explores the possibility that the geometry of space, in certain models, can be described in terms of patterns of entanglement in an underlying quantum system. In some carefully defined models, the amount of entanglement between regions is related to geometric quantities such as the area of surfaces. There are also intriguing mathematical connections between error-correcting codes and these models: some researchers have described parts of the relationship between a model space and its boundary as resembling a quantum error-correcting code. This is serious, active research. It does not yet say that our own space is built this way, and it has no direct bearing on how a quantum chip should be wired. It is fair to say only that the vocabulary of codes, graphs and entanglement turns out to be useful in those studies, which is interesting in itself.

Chapter 10. The Quantum Spatial Mapping Project, Seen from the Chip

The Quantum Spatial Mapping project is a public web page at www.quantumspatial.lol. It is not a laboratory, a company or a research group, and it does not present itself as one. This chapter describes what the page says, as I found it while writing, offers some gentle corrections to its mathematics because the page explicitly asks for them, and then suggests where the project's central question might meet the quantum computing ideas of this book. Everything I say about the project comes from the page itself. Where I go beyond it, I say so.

What the page says

The page is titled “Quantum Spatial Mapping”, with a large heading “Quantum Spatial” and the subtitle “Exploring a new branch of quantum physics”. Its logo is described, in the page's own accessibility text, as “an atom with a question mark at its center”.

Under the heading “Theory”, the page describes the project as “a playful, evolving proposal for thinking about the relationship between space and quantum phenomena.” It goes on: “Rather than treating space as a passive container for quantum events, this framework asks whether spatial geometry might play an active role in mediating quantum interactions. The idea is intentionally open to scrutiny, questions, and better explanations.”

Under “Mathematical Framework”, it introduces three expressions as “a starting point for discussion, building on familiar quantum-mechanical notation”, and then asks: “What do these terms mean here? Which assumptions are missing? That is precisely where constructive responses are welcome.”

Under “We Want Your Thoughts”, it says: “This is an evolving theory. Send a critique, an alternative interpretation, a reference, or simply tell us where the joke stops being funny.” It offers a way to send feedback, a link to open an issue on GitHub, a link to a PDF of quantum teleportation formulas, and a link to the earlier book in this series. A short list headed “Questions to Explore” asks: “Could spatial geometry influence quantum measurement?”, “What predictions would distinguish

this idea from established models?”, “Where do the mathematics need clarification or correction?” and “Is this physics, speculative fiction, or both?” The footer reads “Quantum Spatial Mapping · serious questions, questionable certainty”.

That is the whole of it. The page reports no experiments, data, devices or results, and claims none. Its tone is light, its questions are real, and it asks for help. It seems fair to read it in exactly that spirit: as a prompt, not a theory to be defended or attacked.

The three expressions

The page prints its expressions in plain text. Written in standard notation, they are:

$$H = H_0 + \frac{\hbar^2}{2m} \nabla^2 \Psi + V(r, t)$$

$$T_{\text{spatial}} = \frac{\hbar^2}{2m} \sqrt{\det g} \partial_\mu (\sqrt{\det g} g^{\mu\nu} \partial_\nu \Psi)$$

$$U(t) = \exp(-i H_{\text{spatial}} t / \hbar)$$

The page asks where its mathematics need clarification or correction. Here are a few small points, offered in the way one might mark up a friend’s draft. Each of them is easily fixed, and none of them undermines the general direction.

The sign of the kinetic term. In standard quantum mechanics the kinetic energy operator is $-\frac{\hbar^2}{2m} \nabla^2$, with a minus sign. Without it, the “kinetic energy” would be negative for an ordinary wave, which is not what is intended. The same sign is missing from the second expression.

An operator mixed with a wavefunction. In the first expression, H_0 and $V(r, t)$ are operators, things that act on a state, while $\nabla^2 \Psi$ is what you get after an operator has acted on the state Ψ . Adding them together mixes two kinds of object. The intended meaning is almost certainly

$$H = H_0 - \frac{\hbar^2}{2m} \nabla^2 + V(r, t),$$

with the Schrödinger equation $i\hbar \partial_t \Psi = H\Psi$ then describing how H acts on Ψ . It would also help to say what H_0 contains that the kinetic and potential terms do not; otherwise the kinetic energy may be counted twice.

The curved-space term. The second expression is clearly reaching for something real and well known: the Laplace-Beltrami operator, which is the standard way of writing ∇^2 on a curved surface or in a curved space described by a metric g . In that operator the outer factor of $\sqrt{\det g}$ divides rather than multiplies. With that change and the minus sign restored, the expression reads

$$T_{\text{spatial}} \Psi = -\frac{\hbar^2}{2m} \frac{1}{\sqrt{\det g}} \partial_i (\sqrt{\det g} g^{ij} \partial_j \Psi),$$

which is exactly the kinetic energy of a quantum particle moving in a space with metric g . This is established physics: it is how one writes the Schrödinger equation on a sphere, on a curved surface, or in curved coordinates. I have written the indices as i and j , which by convention run over space only, since the expression has the Schrödinger-style factor $\hbar^2/2m$. The Greek indices μ and ν on the page usually signal space and time together, and it would be worth the project saying which is meant.

An undefined Hamiltonian. The third expression uses H_{spatial} , which the page does not define. The natural reading is $H_{\text{spatial}} = T_{\text{spatial}} + V(r, t)$, possibly with H_0 added, and stating that explicitly would tie the three lines together into one framework.

Time dependence. The formula $U(t) = \exp(-iHt/\hbar)$ is correct for a Hamiltonian that does not change with time. But the first expression includes a potential $V(r, t)$ that does depend on time. In that case the simple exponential has to be replaced by a time-ordered version, often written

$$U(t) = \mathcal{T} \exp \left(-\frac{i}{\hbar} \int_0^t H_{\text{spatial}}(t') dt' \right),$$

where the symbol $\mathcal{T}$ means that the contributions from different moments are applied in the order in which those moments occur. Alternatively, the project could choose a potential that does not depend on time, and then the page's formula would stand as written.

With these adjustments, the framework describes a quantum particle moving in a fixed curved space under a potential. That is a well-understood and respectable starting point. It is also, as it stands, ordinary quantum mechanics on a curved background, which is the honest answer to the page's question about which assumptions are missing: to become something new, the framework would need an extra ingredient, such as a rule by which the geometry itself responds to the quantum state, or a definite claim about how geometry affects measurement.

Where the project meets a quantum computer

From here on, the chapter goes beyond what the page says. These are my suggestions, not the project's claims, and they are speculative in the sense that nobody has carried them out for this project.

The corrected curved-space term is a natural thing to simulate on a quantum computer, and doing so would connect the project's central question with every theme of this book.

To simulate a particle moving through space, the space must first be broken into a grid of points, as is done in any numerical calculation. On such a grid, the Laplace-Beltrami operator becomes a rule that links each point to its neighbours, with weights that depend on the metric. In other words, it becomes a graph: points joined by edges, with the geometry encoded in the weights of the edges. A curved space, discretised, looks very like a weighted coupling map.

That gives a pleasing correspondence, but it needs to be stated carefully. The graph here belongs to the simulated system, not to the chip. The geometry is something put into the calculation as a set of numbers, and the device's job is to compute how a quantum particle would behave on it. The task of laying that graph onto a real processor is precisely the mapping problem of Chapters 3 and 7. A good mapping would place neighbouring grid points of the simulated space on neighbouring qubits of the chip, or encode position in a register of qubits in a way that keeps the needed operations cheap, so that few swaps are required. There are well-established methods for each of these approaches.

The time-dependence point also has a direct counterpart in practice. Quantum computers usually simulate time evolution by breaking it into many short steps and applying the Hamiltonian for each step in turn, a technique known as Trotterisation, after a mathematical result due to Hale Trotter. When the Hamiltonian changes with time, one simply uses the Hamiltonian appropriate to each moment for each step, in order. That is a step-by-step version of the time-ordered exponential above, and it shows that the correction is not a technicality: it is exactly what a simulation would have to do.

A modest, concrete version of the project's question could then be posed in a way a quantum computer could address. Take a small curved surface, discretised into a handful of points. Prepare a wave packet on it. Let it evolve under the corrected Hamiltonian, and measure where it is likely to be found later. Compare with the same calculation on a flat grid. The difference shows, in a clear and checkable way, how geometry shapes the spreading and interference of a quantum wave. None of this would be new physics; it would reproduce what ordinary quantum mechanics predicts, and small versions could be checked on an ordinary laptop. But it would turn the page's idea that geometry plays "an active role in mediating quantum interactions" into a specific, testable statement about a specific system, which is exactly the move the page's own question about distinguishing predictions asks for.

Closing

A quantum computer is a set of physical carriers of information, arranged in some pattern, with only some pairs able to interact directly. That simple fact shapes nearly everything about how such machines are used. It is why a program must be mapped onto a chip, why routing and swap networks exist, why the same calculation performs differently on two machines of the same size, and why device makers argue carefully about grids, hexagons and shuttling zones.

The information itself is not tied to any one place. It can be moved by swaps, carried along by moving atoms, or shared across many qubits by entanglement. In the surface code it is spread across a whole patch of grid, protected precisely because no local disturbance can reach it, while errors show up as defects that can be read like marks on a map. Mapping, in the established senses of the word, means fitting the relations a calculation needs onto the relations a device provides, and recording how well each part of the device performs.

What mapping does not mean, in this setting, is that a quantum processor reveals the shape of space, that entanglement is a hidden wire, or that moving a quantum state can outrun light. Those boundaries are firm. Beyond them lie real open questions about geometry and quantum information, studied seriously in theoretical physics, and playful proposals such as the Quantum Spatial Mapping page, which asks those questions openly and invites correction.

If you take one thing from this book, let it be a way of looking at a diagram of a quantum processor. The dots are carriers, not locations of information. The lines are possibilities, not distances. The colours on a calibration map are the device's weather. And somewhere in software, a careful piece of planning is deciding where each part of a calculation should live, and how it will travel when it needs to meet another.

Further Reading

Books and lecture notes

Michael A. Nielsen and Isaac L. Chuang, *Quantum Computation and Quantum Information* (Cambridge University Press, 2000; 10th anniversary edition 2010). The standard textbook, including quantum circuits and the foundations of quantum error correction.

John Preskill, lecture notes for Physics 219, Quantum Computation (California Institute of Technology), freely available online. The chapters on error correction are especially clear.

Hardware and the current era

John Preskill, “Quantum Computing in the NISQ era and beyond”, *Quantum* 2, 79 (2018).

P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson and W. D. Oliver, “A quantum engineer’s guide to superconducting qubits”, *Applied Physics Reviews* 6, 021318 (2019).

F. Arute and colleagues, “Quantum supremacy using a programmable superconducting processor”, *Nature* 574, 505 (2019). Describes the Sycamore processor and its grid layout.

D. Bluvstein and colleagues, “Logical quantum processor based on reconfigurable atom arrays”, *Nature* 626, 58 (2024).

Mapping, routing and benchmarking

G. Li, Y. Ding and Y. Xie, “Tackling the qubit mapping problem for NISQ-era quantum devices”, Proceedings of ASPLOS 2019. Introduces the SABRE routing heuristic.

I. D. Kivlichan, J. McClean, N. Wiebe, C. Gidney, A. Aspuru-Guzik, G. K.-L. Chan and R. Babbush, “Quantum simulation of electronic structure with linear depth and connectivity”, *Physical Review Letters* 120, 110501 (2018). The fermionic swap network.

E. Magesan, J. M. Gambetta and J. Emerson, “Scalable and robust randomized benchmarking of quantum processes”, *Physical Review Letters* 106, 180504 (2011).

A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation and J. M. Gambetta, “Validating quantum computers using randomized model circuits”, *Physical Review A* 100, 032328 (2019). Introduces quantum volume.

The documentation of open-source compilers such as Qiskit, which explains layout, routing and optimisation stages in practical terms.

Error correction and the surface code

P. W. Shor, “Scheme for reducing decoherence in quantum computer memory”, *Physical Review A* 52, R2493 (1995).

A. Yu. Kitaev, “Fault-tolerant quantum computation by anyons”, *Annals of Physics* 303, 2 (2003).

E. Dennis, A. Kitaev, A. Landahl and J. Preskill, “Topological quantum memory”, *Journal of Mathematical Physics* 43, 4452 (2002).

A. G. Fowler, M. Mariantoni, J. M. Martinis and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation”, *Physical Review A* 86, 032324 (2012). A thorough and widely cited introduction.

C. Horsman, A. G. Fowler, S. Devitt and R. Van Meter, “Surface code quantum computing by lattice surgery”, *New Journal of Physics* 14, 123011 (2012).

B. M. Terhal, “Quantum error correction for quantum memories”, *Reviews of Modern Physics* 87, 307 (2015).

Google Quantum AI, “Suppressing quantum errors by scaling a surface code logical qubit”, *Nature* 614, 676 (2023).

Google Quantum AI and collaborators, “Quantum error correction below the surface code threshold”, *Nature* (published online December 2024).